Day 136 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 4 reference

Watch: DC-GAN Explained! (Connor Shorten) | Fractionally Strided or Transposed Convolution through an example (Aparajita Ojha)

`day136_dcgan.py`, segment by segment, in the order `main()` uses things. The strided convolution, the transposed convolution, batch normalisation, the layer stack and the junction checker are new; the losses and AdamGAN are Day 134's, the judge is Day 116's softmax regression, and the junction images are Day 47's generator, all imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: k = kernel size · s = stride · p = padding · C = channels · H, W = height, width · m = batch size

Segment 0 — Setup

Imports and the thread setting

CODE

```
os.environ.setdefault("OPENBLAS_NUM_THREADS", "4")
```

- `os.environ.setdefault(...)` — set the variable only if it is not set already, and before numpy is imported: OpenBLAS reads it once, at import.

Segment 1 — Convolutions with a stride

im2col: every patch as a row

MATH

$$\text{output size} = (H + 2p - k) / s + 1 \quad (28 + 2 - 4) / 2 + 1 = 14$$

CODE

```
def im2col(Xp, k, s, oh, ow):
    m, C = Xp.shape[:2]; cols = np.empty((m, oh, ow, C, k, k), Xp.dtype)
    for i in range(k):
        for j in range(k):
            cols[:, :, :, :, i, j] = Xp[:, :, i:i + s * oh:s, j:j + s * ow:s].transpose(0, 2, 3,
                1)
    return cols.reshape(m * oh * ow, C * k * k)
```

- `np.empty((m, oh, ow, C, k, k), Xp.dtype)` — space for every patch, not yet filled (empty does not zero it: every entry is written below).
- `Xp[:, :, i:i + s * oh:s, j:j + s * ow:s]` — start:stop:step slicing: every s-th pixel from offset (i, j): the pixel each output position sees at this kernel offset.
- `.transpose(0, 2, 3, 1)` — reorder the axes to (image, row, column, channel), to match the patch array.
- `cols.reshape(m * oh * ow, C * k * k)` — one row per output position, one column per patch value: a convolution is now one matrix product.

col2im: the reverse, adding overlaps

CODE

```
def col2im(dcols, shape, k, s, oh, ow):
    dX = np.zeros(shape, dcols.dtype); d = dcols.reshape(shape[0], oh, ow, shape[1], k, k)
    for i in range(k):
        for j in range(k):
            dX[:, :, i:i + s * oh:s, j:j + s * ow:s] += d[:, :, :, :, i, j].transpose(0, 3, 1, 2)
    return dX
```

- `dX[...] += ...` — += adds: where patches overlap, a pixel collects a contribution from every patch it is in.
- `d[:, :, :, :, i, j].transpose(0, 3, 1, 2)` — the values at one kernel offset, back to (image, channel, row, column).

Conv: forward and backward

MATH

$$Z = cols\ W^T + b,\quad dW = dZ^T\ cols,\quad d\ cols = dZ\ W \to col2im \to dX$$

CODE

```
class Conv:
    def __init__(self, cin, cout, k, s, p, rng):
        self.W = (rng.randn(cout, cin, k, k) * np.sqrt(2.0 / (cin * k * k))).astype(np.float32)
        self.b = np.zeros(cout, np.float32); self.k, self.s, self.p = k, s, p
    def params(self):
        return [(self.W, self.b)]
    def forward(self, X):
        Xp = pad2(X, self.p); H, W = Xp.shape[2:]
        oh, ow = (H - self.k) // self.s + 1, (W - self.k) // self.s + 1
        cols = im2col(Xp, self.k, self.s, oh, ow); self.cache = (X.shape, Xp.shape, cols, oh, ow)
        Z = cols @ self.W.reshape(len(self.W), -1).T + self.b
        return Z.reshape(len(X), oh, ow, -1).transpose(0, 3, 1, 2)
    def backward(self, dZ):
        xs, ps, cols, oh, ow = self.cache; F = len(self.W)
        dZm = dZ.transpose(0, 2, 3, 1).reshape(-1, F)
        self.grads = [((dZm.T @ cols).reshape(self.W.shape), dZm.sum(0))]
        dX = col2im(dZm @ self.W.reshape(F, -1), ps, self.k, self.s, oh, ow)
        return dX[:, :, self.p:self.p + xs[2], self.p:self.p + xs[3]] if self.p else dX
```

- `np.sqrt(2.0 / (cin * k * k))` — He scale: each output adds up cin × k × k inputs.
- `cols @ self.W.reshape(len(self.W), -1).T` — W reshaped to (Cout, Cin · k · k): one matrix product convolves every patch with every filter.
- `(dZm.T @ cols).reshape(self.W.shape)` — the weight gradient: each filter's error times the patches it saw.
- `dX[:, :, self.p:..., self.p:...]` — drop the padding: the gradient for the padded border belongs to no real pixel.

ConvT: the transposed convolution

MATH

$$\text{output size} = (H - 1)\,s - 2p + k \quad 6 \times 2 - 2 + 4 = 14; \quad \langle \text{conv}(X), Y \rangle = \langle X, \text{convT}(Y) \rangle$$

CODE

```
class ConvT:
    def __init__(self, cin, cout, k, s, p, rng):
        self.W = (rng.randn(cin, cout, k, k) * np.sqrt(2.0 / (cin * k * k / (s *
            s)))).astype(np.float32)
        self.b = np.zeros(cout, np.float32); self.k, self.s, self.p = k, s, p
    def params(self):
        return [(self.W, self.b)]
    def forward(self, X):
        m, C, H, W = X.shape; k, s, p = self.k, self.s, self.p
        Ho, Wo = (H - 1) * s + k, (W - 1) * s + k
        Xm = X.transpose(0, 2, 3, 1).reshape(-1, C); self.cache = (Xm, H, W)
        Y = col2im(Xm @ self.W.reshape(C, -1), (m, self.W.shape[1], Ho, Wo), k, s, H, W)
        return Y[:, :, p:Ho - p, p:Wo - p] + self.b[None, :, None, None]
    def backward(self, dY):
        Xm, H, W = self.cache; C = len(self.W)
        cols = im2col(pad2(dY, self.p), self.k, self.s, H, W)
        self.grads = [((Xm.T @ cols).reshape(self.W.shape), dY.sum((0, 2, 3)))]
        return (cols @ self.W.reshape(C, -1).T).reshape(len(dY), H, W, C).transpose(0, 3, 1, 2)
```

- `Xm @ self.W.reshape(C, -1)` — each input pixel times the kernel: a k × k patch for every output channel.
- `col2im(..., k, s, H, W)` — the same col2im as the Conv's backward pass: each patch painted s pixels from its neighbour, overlaps added.
- `Y[:, :, p:Ho - p, p:Wo - p]` — crop p from every side: output (H – 1) s – 2p + k.
- `im2col(pad2(dY, self.p), ...)` — its backward pass is a strided convolution of dY: the two layers are each other's adjoint.

BN: batch normalisation

MATH

$$x' = (x - \mu) / \sqrt{(\sigma^2 + 10^{-5})}, \quad y = \gamma\, x' + \beta$$

CODE

```
class BN:
    def __init__(self, c):
        self.W = np.ones(c, np.float32); self.b = np.zeros(c, np.float32)
    def params(self):
        return [(self.W, self.b)]
    def forward(self, X):
        ax = (0, 2, 3) if X.ndim == 4 else (0,); sh = (1, -1, 1, 1) if X.ndim == 4 else (1, -1)
        mu = X.mean(ax, keepdims=True); inv = 1 / np.sqrt(X.var(ax, keepdims=True) + 1e-5)
        Xh = (X - mu) * inv; self.cache = (Xh, inv, ax, sh)
        return Xh * self.W.reshape(sh) + self.b.reshape(sh)
    def backward(self, dY):
        Xh, inv, ax, sh = self.cache; self.grads = [((dY * Xh).sum(ax), dY.sum(ax))]
        dXh = dY * self.W.reshape(sh)
        return inv * (dXh - dXh.mean(ax, keepdims=True) - Xh * (dXh * Xh).mean(ax, keepdims=True))
```

- `ax = (0, 2, 3) if X.ndim == 4 else (0,)` — for images, average over the batch and every position, one mean per channel.
- `keepdims=True` — keep the averaged axes with length 1, so the result broadcasts against X.

- `self.W, self.b` — gamma and beta, stored under the names Adam expects.
- `inv * (dXh - mean - Xh * mean(dXh * Xh))` — the backward pass: every value's gradient also flows through the batch mean and spread.

Act and Net: activations, reshapes and the stack

CODE

```
class Act:
    def __init__(self, kind):
        self.kind = kind
    def params(self):
        return []
    def forward(self, X):
        if self.kind == "relu":
            self.g = (X > 0).astype(X.dtype)
        elif self.kind == "lrelu":
            self.g = np.where(X > 0, 1, 0.2).astype(X.dtype)
        elif self.kind == "sigmoid":
            self.y = 0.5 * (1 + np.tanh(0.5 * X)); return self.y
        else:
            self.shape = X.shape; return X.reshape(self.kind)
        return X * self.g
    def backward(self, d):
        if self.kind == "sigmoid":
            return d * self.y * (1 - self.y)
        if isinstance(self.kind, tuple):
            return d.reshape(self.shape)
        return d * self.g
class Net:
    def __init__(self, layers):
        self.layers = layers; self.net = [pr for L in layers for pr in L.params()]
    def forward(self, X):
        for L in self.layers:
            X = L.forward(X)
        return X
    def backward(self, d):
        for L in reversed(self.layers):
            d = L.backward(d)
        return [g for L in self.layers if L.params() for g in L.grads], d
```

- `np.where(X > 0, 1, 0.2)` — leaky ReLU's slope: 1 for positive inputs, 0.2 for negative ones.
- `isinstance(self.kind, tuple)` — a tuple kind is a reshape target, such as (-1, 64, 7, 7).
- `for L in reversed(self.layers)` — backward runs through the layers in the opposite order.
- `[pr for L in layers for pr in L.params()]` — a nested comprehension: every layer's (W, b) pairs in one list, the same arrays the layers hold.

dcgan_players

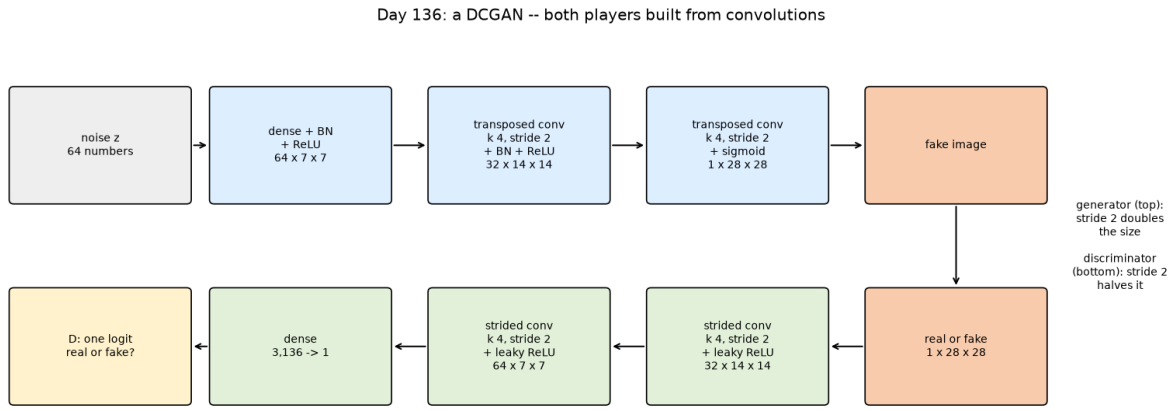
CODE

```
def dcgan_players(seed, k=64, c=32, ch=1, bn=True, out="sigmoid"):
    rg, rd = np.random.RandomState(seed), np.random.RandomState(seed + 1)
    norm = (lambda n: [BN(n)]) if bn else (lambda n: [])
    G = Net([Dense(k, 2 * c * 49, rg)] + norm(2 * c * 49) + [Act("relu"), Act((-1, 2 * c, 7, 7)),
             ConvT(2 * c, c, 4, 2, 1, rg)] + norm(c) + [Act("relu"),
            ConvT(c, ch, 4, 2, 1, rg)] + ([Act("sigmoid")] if out == "sigmoid" else []) +
                [Act((-1, ch * 784))])
    D = Net([Act((-1, ch, 28, 28)), Conv(ch, c, 4, 2, 1, rd), Act("lrelu"),
             Conv(c, 2 * c, 4, 2, 1, rd), Act("lrelu"), Act((-1, 2 * c * 49)), Dense(2 * c * 49,
                 1, rd)])
    return G, D
```

- `norm = (lambda n: [BN(n)]) if bn else (lambda n: [])` — a function returning a list with a BN layer, or an empty list; + joins lists, so 'no BN' simply adds nothing.

- `Act((-1, 2 * c, 7, 7))` — reshape the dense layer's 3,136 numbers into 64 maps of 7 × 7.
- `ConvT(2 * c, c, 4, 2, 1, rg)` — kernel 4, stride 2, padding 1: 7 × 7 becomes 14 × 14.

SYSTEM DIAGRAM



The two convolutional players.

Segment 2 — Training and measuring

net_gan_step: Day 134's round of the game, for layer stacks

CODE

```
def net_gan_step(G, D, oG, oD, X, rng, k):
    n = len(X); F = G.forward(rng.standard_normal((n, k)).astype(np.float32))
    Af = D.forward(F); Ar = D.forward(X)
    ld, gr, gf = d134.d_loss(Ar, Af)
    g1, _ = D.backward(gr); D.forward(F); g2, _ = D.backward(gf)
    oD.step(D.net, [(a[0] + b[0], a[1] + b[1]) for a, b in zip(g1, g2)])
    F = G.forward(rng.standard_normal((n, k)).astype(np.float32))
    lf = D.forward(F); lg, gl = d134.g_loss(lf)
    _, dF = D.backward(gl); gG, _ = G.backward(dF)
    oG.step(G.net, gG)
    return ld, lg, float(d134.d126.sigmoid(Ar).mean()), float(d134.d126.sigmoid(lf).mean())
```

- `Af = D.forward(F); Ar = D.forward(X)` — a layer stack keeps one cache per layer: the second forward pass overwrites the first's.
- `D.forward(F); g2, _ = D.backward(gf)` — so the fake batch is run again before its backward pass.
- `zip(g1, g2)` — pairs the real and fake gradients layer by layer, to be summed.
- `D.backward(gl); gG, _ = G.backward(dF)` — back through D to its input image, then through G: D's own gradients are thrown away.

smoothness: the jump between neighbouring pixels

CODE

```
def smoothness(F, ch=1):
    I = F.reshape(len(F), ch, 28, 28)
    return float((np.abs(np.diff(I, axis=2)).mean() + np.abs(np.diff(I, axis=3)).mean()) / 2)
```

- `np.diff(I, axis=2)` — each pixel minus the one above it; axis=3 compares with the one to its left.

Junctions: the template checker

MATH

misfit = mean over the 33 cross pixels of (image – template)^2; valid if below the real images' 99th percentile

CODE

```
def junction_templates(arm=8):
    T, cross, info = [], [], []
    for r in range(12, 17):
        for c in range(12, 17):
            X = np.zeros((28, 28), np.float32); X[r, c] = 1
            for dr, dc in DIRS:
                X[r + dr * np.arange(1, arm + 1), c + dc * np.arange(1, arm + 1)] = 1
            for m in range(1, 16):
                t = np.zeros((28, 28), np.float32); t[r, c] = 2.0
                for j, (dr, dc) in enumerate(DIRS):
                    if m >> j & 1:
                        t[r + dr * np.arange(1, arm + 1), c + dc * np.arange(1, arm + 1)] += 2.0
                T.append(t.ravel()); cross.append(X.ravel()); info.append(bin(m).count("1"))
    return np.array(T), np.array(cross), np.array(info)
TEMPL, CROSS, ARMS = junction_templates()
def read_junctions(F):
    I = F.reshape(len(F), 3, 784).mean(1)
    best = np.argmin((TEMPL ** 2).sum(1)[None, :] - 2 * I @ TEMPL.T, axis=1)
    misfit = (((I - TEMPL[best]) ** 2) * CROSS[best]).sum(1) / CROSS[best].sum(1)
    return misfit, ARMS[best]
def junction_stats(F, cut):
    err, arms = read_junctions(F); ok = err < cut
    h = np.bincount(arms[ok], minlength=5)[1:] / max(1, ok.sum())
    return float(ok.mean()), h, float(np.median(err))
```

- `for m in range(1, 16)` — the 15 non-empty sets of four arms, one bit per arm.
- `m >> j & 1` — shift right by j and keep the last bit: is arm j in set m?
- `bin(m).count("1")` — the number of 1 bits: how many arms.
- `(TEMPL ** 2).sum(1)[None, :] - 2 * I @ TEMPL.T` — squared distance to every template without $|I|^2$, which is the same for all of them.
- `np.argmin(..., axis=1)` — each image's closest perfect junction.
- `np.median(err)` — the middle misfit of the set: how close a typical image is, even when almost none is valid.

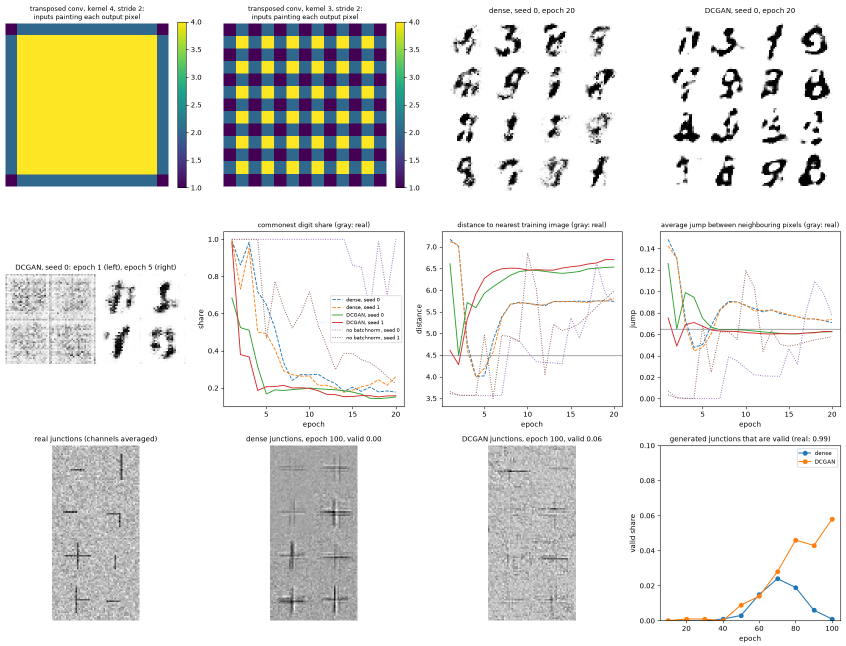
train_watch

CODE

```
def train_watch(G, D, X, judge, epochs, seed, k=64, batch=128, ch=1, every=1, keep=()):
    oG, oD = d134.AdamGAN(G.net, 2e-4, 0.5), d134.AdamGAN(D.net, 2e-4, 0.5)
    rng = np.random.RandomState(seed); n = len(X); Zfix =
        np.random.RandomState(9).standard_normal((2000, k)).astype(np.float32)
    log, snaps, t0 = [], {}, time.perf_counter()
    for ep in range(1, epochs + 1):
        o = rng.permutation(n); hs = []
        for s in range(0, n - batch + 1, batch):
            hs.append(net_gan_step(G, D, oG, oD, X[o[s:s + batch]], rng, k))
        if ep % every == 0 or ep == epochs:
            F = net_generate(G, Zfix); m = np.mean(hs, 0)
            log.append((ep, m[2], m[3], judge(F), smoothness(F, ch), time.perf_counter() - t0))
            if ep in keep:
                snaps[ep] = F[:16].copy()
    return log, snaps
```

- `judge(F)` — a function passed in: digit shares on MNIST, valid share on junctions.
- `if ep % every == 0 or ep == epochs` — % is the remainder: measure every 'every' epochs, and always at the last.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `a[i:j:s]` — slicing with a step: every s-th element from i up to j.
- `x.transpose(0, 2, 3, 1)` — reorder an array's axes.
- `np.where(cond, a, b)` — a where cond holds, b elsewhere.
- `m >> j & 1` — bit j of the integer m.
- `for L in reversed(xs)` — iterate a list backwards.
- `[y for x in xs for y in f(x)]` — a nested comprehension: flatten lists of lists.

End of Day 136 syntax reference. For the experiments and what they showed, see the Day 136 complete guide.